

蓝禾移动电源智能管理产测软件方案

1. 方案概述

本方案针对移动电源产测场景，基于 USB、NFC通信，实现设备与上位机的指令交互、数据采集、产测验证、安全管控等核心功能。整体逻辑围绕「通信协议→产测流程→核心功能→安全机制」展开，兼顾单节 / 多串电池适配性、数据精准性及操作安全性，确保产测高效、可控，同时保障电池使用安全与寿命。

2. 名词解释

名称 / 参数	指令标识	说明
下行指令	上位机→设备	16 进制帧格式，带校验和，用于模式控制与写入
上行指令	设备→上位机	文本行格式，\r\n结尾，用于数据主动上报
帧包头	0xAA 0x55	下行指令固定 2 字节头
校验和	-	包头 + 长度 + 命令 ID + 参数累加和低 8 位
命令 ID	1 字节	指令功能编号
产测模式	0x01+0x01	设备进入测试并上报全部数据
加密签名	8 字节	64 位哈希 (MCU_UID + 密钥)，防非法写入
整机 SN	SN	产品唯一序列号，12 字节 ASCII
电池 SN	BAT_SN	电池芯 / 电池组唯一序列号
芯片唯一 ID	UID	MCU 硬件 ID，8 字节十六进制，只读
总电压	VBAT	电池组总电压 (V)
剩余电量	SOC	电量百分比 (%)
温度	TEMP	电池温度 (°C)
当前内阻	BAT_R	温度补偿后内阻 (mΩ)
初始内阻	BAT_R0	产测合格基准内阻
用户循环	CYCLE_U	可清空显示循环次数（测试OK后清空）
老化循环	CYCLE_A	真实充放次数，不可修改
固件版本	VER	设备固件版本号
机型	MODEL	产品型号编码
生产商	MFG	生产工厂代码，用于溯源
生产日期	MFG_DATE	出厂日期，格式 YYYYMMDD
末次过压	OVP	最近一次过压保护值
1 小时过压峰值	OVP_MAX	1 小时内最高过压记录
末次过温	OTP	最近一次过温保护值
1 小时过温峰值	OTP_MAX	1 小时内最高过温记录
异常时间	ERR_TIME	最近一次异常发生时间
异常次数	ERR_CNT	累计异常次数
内阻异常	R_ERR	1 = 内阻偏大，0 = 正常
UID 异常	UID_ERR	1=UID 读取异常，0 = 正常
进液次数	LIQ_CN	从检测到进液 → 进液消失，计 1 次，掉电保存
通信应答	ACK	指令执行结果，OK/FAIL

3. 通信数据结构

3.1 帧格式

下行指令（上位机→设备）

字段	长度	说明
包头	2	0xAA 0x55
长度	1	不含校验和的总长度
命令 ID	1	功能指令
参数	N	解锁指令固定 8 字节
校验和	1	累加和低 8 位

代码块

```
1 // 计算校验和: data = 数据首地址, len = 前面所有字节长度
2 unsigned char CheckSum(unsigned char *data, int len)
3 {
4     int sum = 0;
5     for(int i=0; i<len; i++)
6     {
7         sum += data[i];
8     }
9     return sum & 0xFF; // 只取低8位
10 }
```

上行指令（设备→上位机）

文本行格式，以 \r\n 结尾，无校验和，包含状态、异常、基础信息。

4. 命令 ID 集

命令 ID	命令名称	功能说明
0x01	模式控制	0x01 进入测试模式，读取数据； 0x02 测试OK（复位退出）：清空用户循环值并退出测试模式； 0x03 测试NG：触发闪灯报警，禁止直接退出，需重新充电后方可下发退出指令
0x20	解锁写入	携带 8 字节哈希签名，验证通过解锁

0x21	写入整机 SN	需先解锁
0x22	写入电池 SN	需先解锁
0x23	同步时间	需先解锁，占6个字节（年、月、日、时、分、秒）10进制编码

5. 通信参数

- 接口：USB或NFC
- 波特率：115200
- 数据位：8
- 停止位：1
- 校验位：无
- 下行：16 进制帧（带校验）
- 上行：文本行（\r\n 结尾）

6. 产测核心流程

6.1 PCBA测试（新增）

适用场景：移动电源主板裸板测试（未组装外壳、电池）

测试连接：产测工装通过顶针连接 PCBA 主板的 TX / RX 测试点。

自动上电上报：PCBA 主板上电后 2 秒内，主动通过 TX 发送 UID 信息，无需上位机请求。上报格式：

代码块

```
1 UID=XXXXXXXXXXXXXXXXX\r\n
```

数据绑定与上传：板厂产测软件**自动捕获 UID**，与本次 PCBA 测试记录（电压、电流、功能等）绑定，形成唯一追溯数据，**实时上传 MES 系统**。

6.2 模式触发与初始化（整机）

- 下发进入测试模式指令：**AA 55 05 01 01 06**
- 移动电源跑马灯显示
- 进入产测模式，自动上报含 MCU_UID 的全部数据

6.3 设备上行数据格式

文本行格式，每行都以\r\n结尾。

代码块

```
1 // 实时状态
2 CELL1=XXX
3 CELL2=XXX
4 CELL3=XXX
5 CELL4=XXX
6 VBAT=XXX
7 SOC=XX
8 TEMP=XXX
9 BAT_R=XXX
10 CYCLE_U=XX
11 CYCLE_A=XX
12
13 // 基础信息
14 UID=XXXXXXXXXXXXXXXXX
15 VER=VX.X.X
16 NAME=XXXX
17 SN=XXXXXXXXXXXXX
18 BAT_SN=XXXXXXXXXXXXX
19 MFG=XX
20 MFG_DATE=YYYYMMDD
21 BAT_R0=XXX
22
23 // 异常记录
24 OVP=XXX
25 OVP_MAX=X.XX
26 OTP=XX.X
27 OTP_MAX=XX.X
28 ERR_TIME=YYYYMMDDHHMM
29 ERR_CNT=XX
30 R_ERR=0
31 UID_ERR=0
32 LIQ_CNT=XX //进液次数
33
34 // 结束应答
35 ACK=OK
```

6.4 解锁写入操作

1. 准备：获取上行上报的 MCU_UID（8 字节）

2. 下行解锁指令（示例）：

AA 55 0D 20 1122334455667788 XX

- 0x0D：长度固定
- 20：命令 ID
- 1122334455667788：8 字节哈希签名

3. 上行反馈：

写入成功返回：ACK=OK\r\n

写入失败返回：ERR=UNLOCK_FAIL\r\n

4. 掉电后权限自动失效，30秒后自动生效，接收到测试Ok后失效，需重新解锁

6.5 产品 SN 写入 (0x21)

指令示例：AA 55 xx 21 50 30 ... 31 38

写入成功返回：SN=字符串\r\n

写入失败返回：ERR=PSN_FAIL\r\n

注：产品 SN 为单组字符串，整体转为 ASCII 十六进制发送

6.6 电池 SN 写入 (0x22)

指令示例：AA 55 xx 22 41 54 4C ... 30 3B 41 54 4C ... 30

写入成功返回：BAT_SN=字符串\r\n

写入失败返回：ERR=BSN_FAIL\r\n

注：电池信息为 ASCII 码格式，单组 / 多组电池 SN 均以完整字符串转十六进制发送；多组电池之间用英文分号分隔，分隔符 ASCII：0x3B

例如：ATL00000009;ATL00000010 整体字符串转 16 进制后填入数据域发送

6.7 日期写入 (0x23)

指令示例

目标时间：2026 年 04 月 15 日 15 时 20 分 30 秒

对应参数：26 04 15 15 20 30

指令示例：AA 55 0A 23 26 04 15 15 20 30 00

写入成功返回：DATE=20 26 04 15 15 20 30\r\n

写入失败返回：ERR=DATE_FAIL\r\n

6.8 退出测试模式

1. 测试OK：下发模式控制指令（命令ID 0x01，参数0x02） AA 55 05 01 02 07

设备校验指令格式合法后，严格按以下顺序执行复位退出流程：

a. 解锁校验与参数保存

权限校验检查设备是否已完成0x20 解锁写入，未解锁则不执行参数保存与重置操作。

■ BAT_R0 初始内阻合法性校验

校验范围：默认值 ±50%

若当前值超出范围，自动纠正为默认值

■ 数据重置与掉电保存

清空用户显示循环值 CYCLE_U

保存校准后的初始内阻：BAT_R0 = BAT_R

保存整机 SN、电池 SN 等配置数据

b. 退出测试模式

完成参数保存后，退出测试模式，设备自动恢复至正常待机状态，退出流程结束。

2. 测试NG：下发模式控制指令（命令ID 0x01，参数0x03） AA 55 05 01 03 08

设备立即触发闪灯报警（四灯全灯，1HZ），此时禁止直接退出测试模式，只有当插入 PD 快充协议充电器时，才自动解除 NG 锁定，退出测试模式。

7. 解锁加密函数

7.1 宏定义（密钥一致）

#define UNLOCK_KEY 0x12345678U

- 需与上位机 GetUnlockCommand 函数密钥完全一致，同一产品确认好之后不再修改。
- 32位无符号十六进制，匹配上位机UInteger类型。

7.2 Secure_Sign 函数（不可修改）

函数原型：uint32_t Secure_Sign(const uint8_t *uid, uint32_t key)

- 与上位机同名函数算法完全一致，生成32位签名，算法禁止修改（确保上下位机签名一致）。
- 参数：uid（8字节十六进制字节流，上位机从设备“UID=”字段提取、补0处理）；key固定传 UNLOCK_KEY。
- 返回值：32位十六进制签名，上位机用于生成解锁指令，MCU仅比对前4字节。

7.3 CheckUnlock 函数（核心：仅比对前4字节）

函数原型：uint8_t CheckUnlock(const uint8_t *uid, uint16_t uid_len, uint32_t unlock_code)

- 功能：校验上位机0x20解锁指令中的8位解锁码，仅比对前4字节，控制解锁后操作权限。
- 参数：unlock_code为上位机下发32位解锁码，仅前4字节有效。
- 上位机解锁相关指令：
 - 解锁指令：AA55开头 + 长度 + 0x20 + 8位解锁码（32位签名转十六进制） + 校验和（MakeFrame自动组帧）。
 - 解锁成功后指令：依次下发0x21（写SN）、0x22（写BATSN，可选）、0x23（同步时间），均自动组帧。

7.4 上下位机交互逻辑

- 上位机：设备上报UID→处理为8字节→生成签名→下发0x20解锁指令。
- MCU：接收指令→提取解锁码→CheckUnlock校验（前4字节）→成功（返回1）可执行后续操作。

完整MCU C语言代码（可直接复制使用）

代码块

```
1  #include <stdint.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  #define UNLOCK_KEY 0x12345678U
6
7  // 这个函数完全不动!
8  uint32_t Secure_Sign(const uint8_t *uid, uint32_t key)
9  {
10     const uint32_t mul1 = 0x45D9F3AB;
11     const uint32_t mul2 = 0x9E3779B9;
12     uint32_t hash = key;
13     for (int i = 0; i < 8; i++) {
14         hash ^= uid[i];
15         hash *= mul1;
16         hash ^= hash >> 17;
17         hash += mul2;
18     }
19     hash ^= hash << 19;
20     hash *= mul2;
21     hash ^= hash >> 23;
22     hash *= mul1;
23     return hash;
24 }
```



```

25
26 // 精简版校验函数
27 uint8_t CheckUnlock(const uint8_t *uid, uint16_t uid_len, uint32_t unlock_code)
28 {
29     uint8_t uid8[8] = {0};
30     if(uid_len >= 8)
31         memcpy(uid8, uid, 8);
32     else
33         memcpy(uid8, uid, uid_len);
34
35     uint32_t local = Secure_Sign(uid8, UNLOCK_KEY);
36     // 关键：仅比对前4个字节（32位签名的高16位，与上位机下发的解锁码前4字节比对）
37     return (((local & 0xFFFF0000) >> 16) == ((unlock_code & 0xFFFF0000) >>
16)) ? 1 : 0;
38 }
39
40 // 主函数：精简 + 同时调用两个函数
41 int main()
42 {
43     const uint8_t uid[8] = {1,2,3,4,5,6,7,8};
44     // 1. 调用 Secure_Sign 生成解锁码
45     uint32_t code = Secure_Sign(uid, UNLOCK_KEY);
46     printf("生成解锁码: 0x%08X\n", code);
47
48     // 2. 调用 CheckUnlock 校验
49     uint8_t ok = CheckUnlock(uid, 8, code);
50     printf("ACK=%s\n", ok ? "成功" : "失败");
51
52     // 测试错误码
53     ok = CheckUnlock(uid, 8, 0x12345678);
54     printf("ERR= %s\n", ok ? "成功" : "失败");
55     return 0;
56 }

```

8. 移动电源软件相关逻辑

8.1 USB口进液检测与腐蚀抑制方案

(1) 目的

通过检测 USB-C 接口进液漏电状态，识别 CC 与 D+ 导通行为，动态控制 CC 输出，抑制电解腐蚀，降低 C 口功能不良率。

(2) 硬件架构（按 MCU 是否带 PD 区分）

架构 1：MCU 自带 PD 协议

- MCU 直接控制 CC、D+，可输出10-50Hz 检测脉冲；
- 无需外部 MOS 管。

架构 2：MCU 无 PD 协议（必须通过 MOS 管控制 CC）

- CC 由充电芯片输出 10-50Hz 脉冲；
- MCU 无 PD 协议，必须通过外部 MOS 管将 CC 拉到 GND，实现关断控制；
- MCU 通过 10K 电阻 + IO 监控 D+ 边沿；
- MCU 不参与 PD 通信，仅做进液检测与保护。

(3) 检测使能条件（通用）

- 设备进入待机 / 低功耗模式
- 非充电、非放电状态
- 无外接设备、无数据线连接

(4) 进液检测软件方案

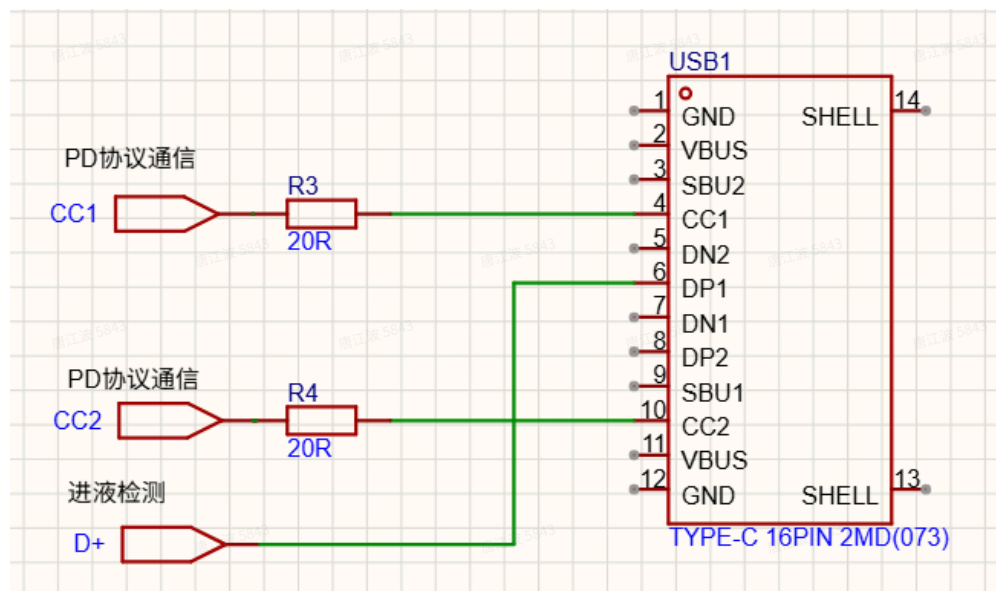
核心检测执行逻辑

架构 1：自带 PD 协议 MCU

直接在 CC 引脚电平切换时 同步检测 D+ 电平；

若 CC 与 D+ 电平连续同步跳变累计达到 10 次 → 判定接口进液；

检测更精准、无延时、不依赖定时器。



架构 2：MCU 无 PD 协议

D+ 出现上升沿 → 中断唤醒 MCU；

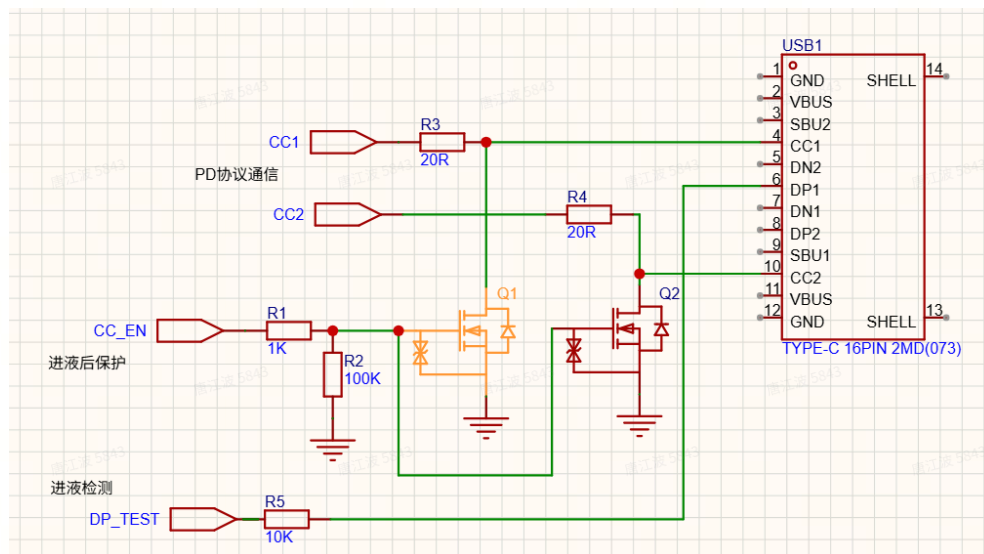
MCU 退出待机，延时保持 0.2s 不进入待机；

在此窗口内用定时器记录两次 D+ 中断间隔 Δt ；

若 Δt 在 10ms~200ms 范围内 → 记 1 次有效同步；

有效累计达到 10 次 → 判定接口进液；

干扰、间隔不匹配、脉冲丢失 → 不计数、不清零、不触发。



注：进液 = CC 通过漏电耦合到 D+，因此两种方案均**无需检测 CC 脚电压**，软件可稳定实现。

(5) 进液后腐蚀抑制策略

- 判定进液后：**MCU 将 CC 脚拉低到 GND，保持 1 分钟**，彻底切断电解腐蚀回路；
- 1 分钟后释放 CC，100ms，确保有一个脉冲输出；
- 这样占空就会降至 1/600，抗腐蚀能力提升 500 倍以上；
- 充放电过程中关闭检测，避免影响到充电兼容性；
- 进液次数 (LIQ_CN)：从检测到进液 → 进液消失，计 1 次，掉电保存。

(6) 进液复检逻辑

架构 1：自带 PD 协议

- 进入保护后，CC 引脚切换时检测到 **1 次 CC 与 D+ 高低电平同步** → 判定进液持续；
- 连续 **2 次检测到 CC 与 D+ 高低电平不同步** → 视为液体消失，退出保护。

架构 2：MCU 无 PD 协议

- 进入保护后，100ms 内检测到 1 次 D+ 上升沿 → 判定进液持续，立即关闭 CC 脚；
- 连续 200ms 未检测到 D+ 上升沿 → 视为液体消失，退出保护。

(7) C口进液保护测试

测试项目	测试目的	测试条件	样品数(pcs)	测试步骤	
C口进液保护测试	模拟用户在握持移动电源使用等场景，保护产品避免汗液等进入C口短路带来风险。	满电产品，自来水	2	1.将满电样品的C口，用C-C测试治具接上，测试治具TypeC母座一端放置自来水中4个小时； 2.中途录视频观察样品是否出现指示灯异常和线头端子是否有腐蚀，中途不允许线从移动电源C口拔出来。	1.拿出泡在座内部Pin用吹风机吹后，能够正常（另一端不接）；2.接以正常充电闪烁等异常

断充测内阻法

触发条件：

8.1.1 未曾计算过内阻值：环温>9℃，充电至 SOC>30% 时自动触发；(保证产线能读取内阻值)

8.1.2 内阻值正常：环温>9℃，SOC<40% 再充电至 SOC>60% 时自动触发。

执行逻辑：充电中短暂关断充电 3 秒（用户无感），再恢复充电

计算公式：内阻 (mΩ) = (断充前电压 - 断充后 2 秒电压) ÷ 充电电流

注：计算完成后，内阻结果保存至 BAT_R，便于实时读取调用

有效条件：

充电电流 > 1A

ADC 采样取平均值

仅计算 1 次，不重复触发

温度补偿算法

基准温度：25℃ → 补偿 0%

温度>25℃：不补偿

温度≤25℃补偿比例：24℃→-2%，23℃→-4%，22℃→-6%，21℃→-8%20℃→-10%，19℃→-12%，18℃→-15%，17℃→-18%16℃→-21%，15℃→-23%，14℃→-26%，13℃→-30%12℃→-36%，11℃→-43%，≤10℃→-51%

优点（精度极高、可控性极强）

充电电流固定、已知、精准

断充瞬间电压跌落 = 真实内阻压降

软件自动控制，不依赖用户行为

2~3 秒极短，用户完全无感

可强制触发、可重复、精度稳定

非常适合量产校准

用户使用过程中电池有异常，可及时限制充电

缺点

无明显缺点，仅需软件控制充电 MOS 短暂关断

8.2 充电保护限制策略

目的：电池鼓包、析锂、漏液均会导致电池内阻升高，通过分级限制充电电压、电流或直接关断充电，可有效抑制异常恶化，降低安全风险。

(1) 常规充电限制

触发条件（满足任一即可）

- 实时内阻 $BAT_R \geq BAT_R0 \times 1.8$
- 循环次数 ≥ 300 次

保护动作：

- 单节限压：最高电压 - 0.2V
- 电流限制： $\leq 0.5C$

解除条件：满足环温 $> 9^{\circ}C$ ，且从 SOC $< 40\%$ 充电至 SOC $> 60\%$ ，重新计算内阻后可解除锁定

(2) 严重内阻异常关断保护

触发条件（需同时满足）

- 正在充电
- 环温 $> 9^{\circ}C$
- SOC $> 30\%$
- 实时内阻 $BAT_R > BAT_R0 \times 2.5$

执行动作

- 立即执行内阻复测，结果更新至 BAT_R

- 若复测后 $BAT_R \geq BAT_R0 \times 2.5$ 立即关闭电池充电四灯全亮，1Hz 闪烁提示异常

限制规则

- 拔掉充电器之前，只执行一次测试，不重复触发

8.3 异常保护策略

- 过温超过 90°C ，永久锁死充电（四灯全亮，1Hz 闪烁提示温度异常）
- 过压 > 20 次，永久锁死充电（四灯全亮，1Hz 闪烁提示异常）
- 任一接口检测到进液后，在进液保护解除前，该接口禁止充放电（不作提示）

8.4 循环计数逻辑

- Cycle_A：真实循环次数，不可改，掉电保存
- Cycle_U：用户显示循环次数，可清空
- 校准公式： $Cycle_U = \text{if}(Cycle_A - 20 > 0, Cycle_A - 20, 0)$;
- 超差计算案例

Cycle_A = 125: $125 - 20 > 0 \rightarrow \text{Cycle_U} = 105$

Cycle_A = 15: $15 - 20 < 0 \rightarrow \text{Cycle_U} = 0$

Cycle_A = 30: $30 - 20 > 0 \rightarrow \text{Cycle_U} = 10$

附：锂电鼓包内阻变化一、典型数据（锂电池）

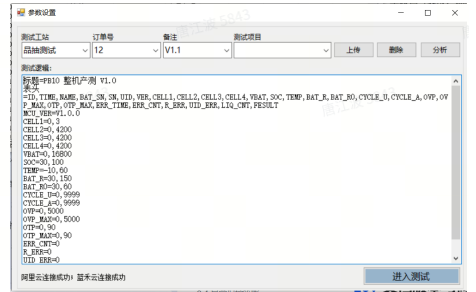
- 轻微鼓包（肉眼刚看出）内阻增：30%~60%例：新电池 $35\text{m}\Omega \rightarrow$ 鼓包后 $45\sim 55\text{m}\Omega$
- 明显鼓包（凸起、手感硬）内阻增：80%~150%例： $35\text{m}\Omega \rightarrow 60\sim 80\text{m}\Omega$
- 严重鼓包（变形、漏液风险）内阻可超 200%+，甚至超出测试仪量程
- 动力 / 储能电芯：正常 $\sim 0.2\text{m}\Omega \rightarrow$ 鼓包后 $> 0.5\text{m}\Omega$

9. 上位机操作说明

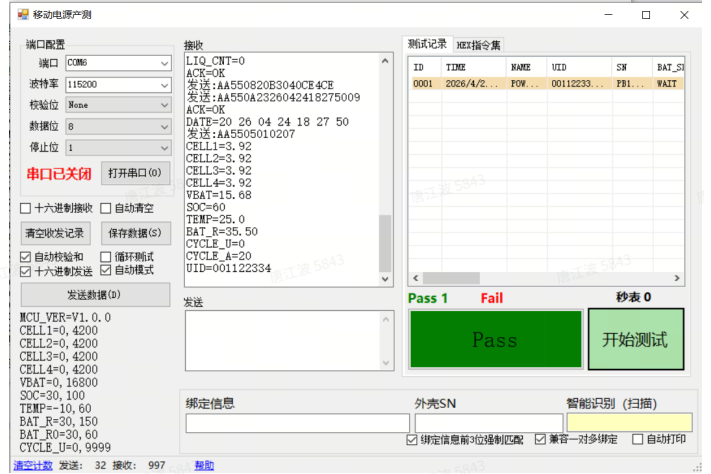
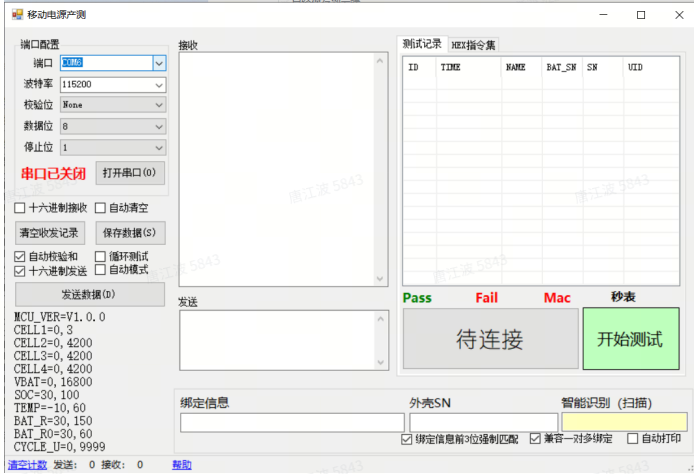
1. 输入账号密码>>功能测试>>移动电源产测>>参数配置>>进入测试

临时账号：开发测试

临时密码：KF2026



2. 选择端口>>扫描SN>>开始测试



10. 产线流程



11. 文档版本与修订记录

版本号	编制日期	修订人	更新内容说明
V1.0	2026/4/5	彭希遴	初始完整版编制；完整定义通信帧格式、命令 ID 集、产测全流程、MC 加密算法、USB-C 进液检测、内阻测算、分级充电保护、循环计数及充电全套逻辑
V1.0	2026/4/24	彭希遴	1、修改校验和错误 AA 55 05 01 01 07 改成 AA 55 05 01 01 06 AA 55 05 01 02 08 改成 AA 55 05 01 01 07 AA 55 05 01 03 09 改成 AA 55 05 01 01 08 2、未曾计算过内阻值：环温>9℃，充电至 SOC>30% 时自动触发； 3、对解锁加密函数进行了补充

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

(注：内容由 AI 生成，请谨慎参考)

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843

唐江波 5843